

API Producción Virgilio → ISIS

Pedidos armados

Cliente: Loekemeyer Hnos. SRL · **Ticket ISIS:** 1159666
Documento: respuesta al Punto 18 (“Próximos pasos Cliente”) del informe consolidado
Versión de la API: 2.0 · **Fecha:** 04/09/2026

Lo que pide el Punto 18 es la API para que ISIS se conecte con nosotros: URL, token, endpoints y estructura. Está todo acá, **menos el token**, que se envía por canal aparte (§3).

El flujo es en un solo sentido: de Virgilio hacia ISIS. No le pedimos a ISIS que nos devuelva ningún dato — ver §5.

0. Nuestra pipeline, y en qué momento entra ISIS

El pedido se construye **entero** del lado de Virgilio, y recién al final se lo pasamos a ISIS:

- 1 **Se toma el pedido** — en la página web de Loekemeyer o de Chef, o por el vendedor.
- 2 **Se numera y, si hace falta, se parte.** Un pedido grande se divide en varias entregas según el volumen que entra en el camión. Cada parte lleva una **referencia nuestra** (ej. LK 1344).
- 3 **Se programa** — se le asigna fecha de entrega y tanda de armado, agrupando por zona de reparto.
- 4 **Se arma en el depósito** — picking, armado y control.
- 5 **Queda cerrado**, y ahí se publica en esta API con los códigos y las cantidades **definitivos**.
- 6 **ISIS lo da de alta y lo factura.**
- 7 **Nosotros vinculamos la factura con el pedido**, de nuestro lado. ISIS no tiene que avisarnos nada.

Lo que ISIS recibe, entonces, no es un pedido a medio armar ni un informe de producción: es un **pedido**, con lo que realmente salió del depósito, listo para facturar.

Qué cambia respecto de hoy

	Hoy	Con esta API
Qué se manda	el pedido tal como se tomó, todavía sin armar	el mismo pedido, ya armado
Cuándo	mail de las 12:30	cuando el pedido queda armado y cerrado
Cómo llega a ISIS	se carga a mano desde ese mail	ISIS lo baja de esta API
Formato y campos	los de hoy	los mismos
Cantidades y códigos	los pedidos	los realmente preparados

O sea: para ISIS el pedido es el mismo tipo de documento de siempre, con los mismos campos. Cambia **cuándo** llega, **por dónde** llega, y que las cantidades ya vienen firmes.

1. Infraestructura: no hace falta Windows Server, IIS, IP pública ni abrir puertos

El Punto 18 pide confirmar la infraestructura para “operar mediante API” y consultar la configuración de Windows Server / IIS / IP pública / puertos. Eso corresponde al escenario en el que **nosotros llamamos a la API del ISIS on-premise** (mensaje de Horacio Barbieri del 25/08). Se resuelve por el otro camino, el que plantea el §11 del propio informe:

ISIS consulta una API nuestra. El servicio corre en la nube, con HTTPS y certificado válido. ISIS hace requests **salientes** desde su red, con la frecuencia que quiera.

Requisito del Punto 18	Estado
Windows Server	No se necesita. El servicio es cloud.
IIS	No se necesita.
IP pública fija del depósito	No se necesita.
Abrir puertos / firewall expuesto	No se necesita. No se expone nada de la red del depósito.
Disponibilidad 24/7	La da la plataforma cloud, no un servidor del depósito.

Del lado de ISIS sólo hace falta **salida HTTPS (puerto 443)** hacia el dominio del servicio.

2. URL del servicio

`https://hrxfctzncixxqmpfhskv.supabase.co/functions/v1/isis-api`

De acá en más, {BASE} = esa URL. Todos los endpoints aceptan además el prefijo opcional /v1 ({BASE}/v1/pedidos es lo mismo que {BASE}/pedidos).

Healthcheck para probar conectividad y token:

```
curl -H "X-API-Key: <TOKEN>" {BASE}/ping

{ "ok": true, "servicio": "Producción Virgilio – API pedidos armados",
  "version": "2.0", "cliente": "Sistemas ISIS - producción",
  "hora": "2026-09-04T14:07:21.137Z" }
```

3. Credenciales

Autenticación por **token fijo** en el header X-API-Key: <TOKEN>. También se acepta Authorization: Bearer <TOKEN> para clientes que no permiten headers arbitrarios.

- El token se entrega **por canal aparte** (no va en este documento).
- Es un token de producción, de un solo consumidor (Sistemas ISIS).
- Se puede revocar y reemplazar en cualquier momento sin tocar el servicio.

- Del lado de Virgilio se guarda **sólo el hash SHA-256**: si se pierde, no se recupera — se emite uno nuevo.

Sin token o con token inválido, cualquier endpoint responde **401**.

4. Endpoints

Son **dos, los dos de lectura**. Cada pedido se identifica con una **referencia nuestra** (referencia), del estilo LK 1344 o CH 812. Es sólo la clave para listarlo y bajarlo: **no es una NP de ISIS ni pretende serlo**, y no viaja dentro del cuerpo del pedido.

4.1 GET {BASE}/pedidos — listado de pedidos armados

Devuelve las **cabeceras** de los pedidos en un estado dado. Es el endpoint de polling: ISIS lo llama cada N minutos y se lleva los que están en pendiente.

Parámetro	Default	Valores
estado	pendiente	pendiente, entregado, anulado
empresa	(todas)	LK (Loekemeyer) o CH (Chef)
desde	(sin filtro)	fecha/hora ISO-8601; devuelve lo terminado desde ese momento
limit	100	1 a 500

```
curl -H "X-API-Key: <TOKEN>" "{BASE}/pedidos?estado=pendiente&limit=50"

{ "ok": true, "estado": "pendiente", "total": 1,
  "pedidos": [
    { "referencia": "LK 1344", "empresa": "LK", "estado": "pendiente",
      "cod_cliente": "288", "razon_social": "MEGA BAZAR S.A.",
      "fecha_entrega": "2026-09-04",
      "terminado_en": "2026-09-03T16:35:20-03:00",
      "items": 14, "cajas": 25 }
  ] }
```

4.2 GET {BASE}/pedidos/{referencia} — el pedido completo

Devuelve el pedido con el formato de la §6, listo para darlo de alta. **Al responder, el pedido pasa de pendiente a entregado** — así ISIS no lo vuelve a levantar en el siguiente ciclo de polling, sin necesidad de confirmar nada. Para consultarlo sin cambiar el estado: ?marcar=false.

```
curl -H "X-API-Key: <TOKEN>" "{BASE}/pedidos/LK%201344"
```

4.3 Errores

```
{ "ok": false, "codigo": "token_invalido", "error": "Token inválido o dado de baja." }
```

HTTP	codigo	Cuándo
401	sin_token / token_invalido	Falta el header o el token no sirve
400	referencia_invalida, estado_invalido	Parámetros mal
404	no_encontrado	La referencia no está publicada
404	ruta_desconocida	URL mal escrita
500/503	error_interno / backend	Error del servicio — reintentar

5. Lo que NO le pedimos a ISIS

Nada vuelve de ISIS hacia Virgilio. No hay endpoint de acuse, no hay que avisar que se facturó, no hay que devolvernos el número de NP ni el comprobante ni el CAE.

El vínculo entre la factura que emite ISIS y el pedido que le mandamos lo resolvemos **de nuestro lado**, con nuestro propio proceso de conciliación sobre los comprobantes. Ya está andando y hoy resuelve la totalidad de las notas de pedido facturadas sin que ISIS haga nada.

Eso deja sin objeto el punto **P3** del informe (“cómo devuelve ISIS el número de NP cuando el pedido nace del otro lado”): no hace falta resolverlo, ni ahora ni después.

Del lado de ISIS el trabajo es **sólo consumir los dos GET de la §4.**

6. Estructura del pedido

Los mismos campos que hoy viajan en el mail de las 12:30, dentro de un sobre con la referencia y la empresa. **El pedido en sí (pedido) no lleva NP.**

```
{
  "referencia": "LK 1344",
  "empresa": "LK",
  "estado_integracion": "entregado",
  "terminado_en": "2026-09-03T16:35:20-03:00",
  "fecha_entrega": "2026-09-04",
  "pedido": {
    "source": "Virgilio",
    "cod_cliente": "288",
    "sucursal_entrega": "Rivadavia 3663 - Mar Del Plata",
    "vend": "7",
    "condicion_pago": "Pago Contado: 25% Dto",
    "condicion_pago_code": 8,
    "payment_term": 2,
    "observaciones": "",
    "items": [
      { "cod_art": "321", "cajas": 6, "uxb": 12 }
    ]
  },
  "control": {
    "items": 14, "cajas": 25, "m3": 0.142,
    "neto_estimado": 775560.24, "moneda": "ARS",
    "nota": "Informativo, sólo para control. El importe a facturar lo determina ISIS con su lista de precios."
  }
}
```

Sobre (uso de la integración, no del pedido)

Campo	Tipo	Descripción
referencia	string	Clave del pedido en esta API (LK 1344). Se usa en la URL y en el listado. No es una NP.
empresa	"LK" "CH"	Razón social del pedido — las dos que contempla el §14 del informe.
estado_integracion	string	pendiente, entregado, anulado.
terminado_en	ISO-8601 -03:00	Momento en que el pedido quedó armado y cerrado.
fecha_entrega	YYYY-MM-DD	Fecha de salida prevista del camión.
anulado	booleano	Aparece sólo si el pedido se dio de baja después de publicarse (§7).

pedido — el pedido propiamente dicho, con los campos de siempre

Campo	Tipo	Descripción
source	string	Origen del pedido. Siempre "Virgilio".
cod_cliente	string	Código de cliente en ISIS.
sucursal_entrega	string	Domicilio de entrega elegido.
vend	string	Código de vendedor.
condicion_pago	string	Condición de pago, en texto.
condicion_pago_code	entero	Código de la condición de pago.
payment_term	entero	Plazo de pago.
observaciones	string	Observaciones del pedido (puede ir vacío).
items[]	array	Las líneas. Ver abajo.

pedido.items[]

Campo	Tipo	Descripción
cod_art	string	Código de artículo realmente preparado . Es el que se factura.
cajas	entero	Cajas realmente preparadas . Es lo que se factura.
uxb	entero	Unidades por caja del artículo.

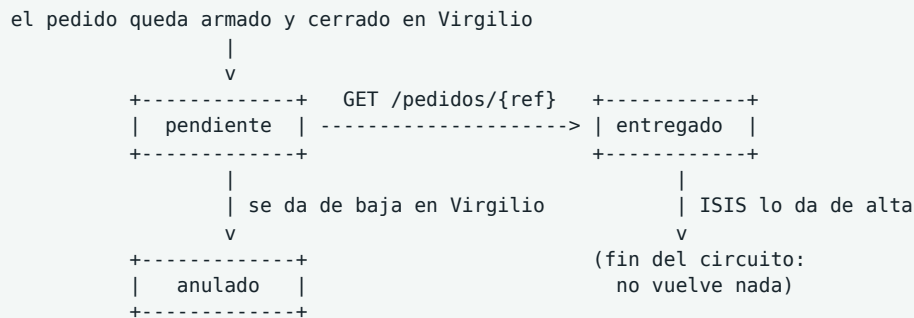
control es **informativo** y no forma parte del pedido. neto_estimado es el neto que calcula Virgilio con su lista de precios; sirve sólo para cotejar contra lo que emite ISIS. m3 son los metros cúbicos, para logística.

Dos reglas importantes de lectura

1. **El pedido no trae NP y no hay que buscarle una.** Se da de alta como se da de alta hoy el pedido del mail: la NP la pone ISIS.
2. **cajas y cod_art son lo preparado, y es lo único que viaja.** No hay campos de “pedido original”: si se preparó menos o se preparó un equivalente, la corrección ya vino hecha. No hay nada que conciliar del lado de ISIS.

Si algún campo que ISIS necesita no está en la lista de arriba, lo agregamos — avisando cuál, sale en el día.

7. Ciclo de vida de un pedido



- **pendiente** — listo para que ISIS lo baje.
- **entregado** — ISIS ya lo bajó. Ahí termina: no hace falta confirmar nada.
- **anulado** — el pedido se dio de baja en Virgilio. Si eso pasa **después** de que ISIS lo bajó, aparece con "anulado": true y se lista con GET /pedidos?estado=anulado. **Conviene consultar ese listado en cada ciclo de polling**, para no facturar algo dado de baja.

Polling sugerido

1. GET /pedidos?estado=pendiente cada 5 a 15 minutos.
2. Por cada referencia: GET /pedidos/{referencia}, y dar de alta el pedido.
3. GET /pedidos?estado=anulado para detectar bajas.

Reintentos: los dos endpoints son seguros de reintentar. Si un GET /pedidos/{referencia} se corta a mitad de camino, se puede volver a pedir — el pedido sigue disponible con el mismo contenido.

8. Trazabilidad

Del lado de Virgilio queda registrado, por referencia: cuándo se terminó de armar, cuándo lo bajó ISIS y cuántas veces, y el JSON exacto que se entregó. Además se loguea cada request de la API (método, ruta, referencia, código HTTP, tiempo de respuesta). Es lo que pide el §18 del informe.

9. Lo que queda pendiente, aparte de esto

Ninguno de estos puntos bloquea lo de arriba:

1. **P4 — Recepción de mercadería (remitos de compra)**. Requiere definir el mapeo de códigos de proveedor y de artículo entre los dos sistemas.
2. **Cutover de la emisión**. Hoy Producción Virgilio emite Factura A por punto de venta 11. Cuando ISIS empiece a facturar estos pedidos hay que **apagar la emisión propia el mismo día**, para no pedir dos CAE por la misma venta. Definir fecha y si el PV 11 queda de respaldo.